
AdaptMath

Adaptivní vzdělávání pro matematické disciplíny

Technická dokumentace

Task Checker — nástroj pro plnění a kontrolu úloh

Projekt: TQ23000206 (TA ČR)
Hlavní řešitel: Mgr. Jana Medková, Ph.D.
Pracoviště: Fakulta informatiky a managementu, Univerzita Hradec Králové
Doba řešení: 09/2025 – 02/2029
Autor dokumentace: Ing. Dominik Palla, Ph.D.
Verze: 1.4
Datum: 24. srpna 2026

Obsah

1	Úvod	3
1.1	Cíl projektu	3
1.2	Co řeší tato dokumentace	3
1.3	Tým	3
2	Architektura	3
2.1	Technologický stack	3
2.2	Struktura repozitáře	4
2.3	Tok dat	4
3	Datový model	4
3.1	Tabulka <code>math_tasks</code>	4
3.2	Konvence <code>task_id</code>	5
3.3	Struktura <code>results</code>	5
3.4	Typy výsledků	5
4	Vektor znalostí a kategorie	5
4.1	Dvě úrovně anotace úlohy	5
4.2	KNOWLEDGE_WEIGHTS (65 vah, verze 260820)	6
4.3	TASK_CATEGORIES, TASK_PROPERTIES, TASK_TYPES, TASK_SKILLS	7
4.4	Kompletní seznam vah	7
4.5	Změny 2026-07-31 (v1.2)	9
4.6	Změny 2026-08-20 (v1.3)	10
4.7	Změny 2026-08-24 (v1.4): Pilotní IRT seed	12
5	Webové UI: Task Checker	12
5.1	Routes	12
5.2	Přihlášení a session	13
5.3	Seznam úloh (<code>/tasks</code>)	13
5.4	Editor úlohy (<code>/tasks/<id></code>)	13
5.5	Tipy pro vyhodnocení MathLive Compute Engine	15
6	Datový obsah	15
6.1	Zdroj úloh	15
6.2	Pokrytí	15
6.3	Distribuce typů výsledků	16
7	Spuštění lokálně	16
7.1	Závislosti	16
7.2	První spuštění	16
7.3	Konfigurace přes env vars	16
7.4	Opětovné nasazení po změně schématu	17
8	Nasazení do produkce (UHK)	17
8.1	Architektura nasazení	17
8.2	Systemd unit	17
8.3	Provozní příkazy	18
8.4	Známá omezení	18

9 Co dál (roadmap)	18
9.1 Krátkodobé	18
9.2 Střednědobé	19
9.3 Dlouhodobé	19
10 Reference	19

1. Úvod

1.1. Cíl projektu

AdaptMath je projekt aplikovaného výzkumu TA ČR (TQ23000206, program SIGMA), jehož cílem je navrhnout a pilotně ověřit nové přístupy k adaptivnímu vzdělávání v matematických oborech na vysokých školách. Klíčovým prvkem je vytvoření *adaptivního engine*, který na základě profilu studenta, klasifikace úloh a jeho odpovědí dynamicky přizpůsobuje výběr další úlohy a tvorbu personalizovaného kurzu.

Plánované výstupy projektu:

- **TQ23000206-V2** — adaptivní engine pro matematické kurzy (software v Pythonu),
- **TQ23000206-V3** — adaptivní kurzy pro výuku matematiky (integrace v Moodle).

1.2. Co řeší tato dokumentace

Dokumentace popisuje **Task Checker** — pomocnou webovou aplikaci určenou pro fázi *plnění a kontroly obsahu*. Aplikace umožňuje výzkumnému týmu:

- importovat matematické úlohy ze zdrojových `.tex` skript do databáze,
- vizuálně procházet a ověřovat zadání i jejich očekávané výsledky,
- editovat všechny atributy úlohy (zadání, výsledky, kategorie, vektor vah, IRT parametry),
- testovat odpovědi pomocí **MathLive Compute Engine** stejným způsobem, jakým je bude vyhodnocovat budoucí studentský engine.

Adaptivní engine (IRT, BKT, personalizace) zatím není součástí — bude přidán v následující fázi projektu.

1.3. Tým

Člen	Role	Odpovědnost
dr. Jana Medková	hlavní řešitel	vize datového modelu, vedení projektu
dr. Jiří Haviger	IRT engine	<code>py-irt</code> , kalibrace obtížnosti
dr. Eva Ševčíková	BKT modely	<code>py-bkt</code> , učící parametry α, β
dr. Petr Bauer	obsah	spoluauditor skriptu <i>Základy matematiky 1</i>
dr. Štefan Šteflovíč	člen týmu	
Ing. Dominik Palla, Ph.D.	vývoj	frontend, integrace, LLM funkce

2. Architektura

2.1. Technologický stack

Vrstva	Technologie
Backend	Python 3.10+, Flask 3.1, SQLAlchemy 2.x, psycpg2
Databáze	PostgreSQL 15 (v Dockeru)
Frontend (render)	KaTeX 0.16 — sazba matematiky
Frontend (vstup)	MathLive (<code><math-field></code> web component)
Vyhodnocení odpovědí	MathLive Compute Engine (LaTeX → MathJSON)
Šablony	Jinja2 (server-side render)
Kontejnerizace	Docker Compose
Frontend libraries	CDN (jsdelivr, unpkg) — žádný build krok

2.2. Struktura repozitáře

```

adapt_math/
  app.py                # Flask aplikace (routes, API)
  database.py           # init_db (SQLAlchemy engine + session)
  model.py              # ORM model MathTask
  seed_db.py            # drop & recreate + napln. uloh
  docker-compose.yml    # PostgreSQL kontejner
  tasks/
    __init__.py         # def. uloh, jeden soubor = jedno cviceni
    knowledge_weights.py # agreguje ALL_TASKS
    cv01.py ... cv13.py # ulohy ze skripta "Zaklady matematiky 1"
  templates/
    tasks_list.html     # seznam vseh uloh (filtr)
    task_checker.html    # editor jedne ulohy + sandbox

```

2.3. Tok dat

1. Zdrojové .tex skripty (cvičení 1–13) jsou ručně přepsány do strukturovaných Python slovníků v `tasks/cvNN.py` (LLM-asistovaná extrakce, viz sekce 6).
2. `seed_db.py` smaže a znovu vytvoří tabulku `math_tasks` a vloží do ní všech 415 úloh přes SQLAlchemy.
3. `app.py` (Flask) servíruje webové UI; přes endpoint `POST /api/tasks/<id>` expert ukládá změny.
4. Frontend MathLive Compute Engine vyhodnocuje studentské odpovědi symbolicky / numericky, bez backend kola.

3. Datový model

3.1. Tabulka `math_tasks`

Definováno v `model.py` (SQLAlchemy ORM). Aktuálně jediná tabulka.

Sloupec	Typ	Popis
<code>task_id</code>	String (PK)	Identifikátor stylu <code>cvNN_i</code> (např. <code>cv04_5</code>).
<code>content_latex</code>	String	Zadání úlohy v LaTeXu; KaTeX render. Pure-math (bez \$) se v UI automaticky obalí do <code>\$\$...\$\$</code> .
<code>results</code>	JSON	Pole pojmenovaných výsledků (viz 3.3).
<code>category</code>	String	Primární téma úlohy — jeden řetězec z 45 <code>TASK_CATEGORIES</code> (jen SŠ + VŠ matematika, bez vlastností/typu/dovedností).
<code>properties</code>	JSON	Multi-select tagy: list ze 9 <code>TASK_PROPERTIES</code> (Vlastnosti — lineární, kvadratická, mocninná, ...).
<code>task_type</code>	JSON	Multi-select tagy: list z <code>TASK_TYPES</code> (zatím 1 položka — <i>Aplikační</i>).
<code>skills</code>	JSON	Multi-select tagy: list ze 6 <code>TASK_SKILLS</code> (Dovednosti — aplikace vzorce, vytýkání, roznásobení, výpočet rovnic/nerovnic, derivování).
<code>knowledge_vector</code>	JSON	Slovník <code>{name: int 0-100}</code> ; podíl všech 61 skill-komponent pro distribuci přírůstku znalosti. Ukládají se jen non-zero hodnoty.
<code>cognitive_load</code>	String	Klasifikace A–F (typ / praktičnost úlohy).

<code>graph_vector</code>	JSON	<i>Deprecated.</i> Starší tag-vektor (list stringů). Zachováno pro kompatibilitu, v UI se needituje.
<code>irt_difficulty</code>	Float	IRT obtížnost (± 3). Doladí Haviger.
<code>irt_discrimination</code>	Float	IRT diskriminace ($\pm 2,5$).

3.2. Konvence `task_id`

Formát `cvNN_i`, kde:

- `NN` je dvouciferné číslo cvičení (01–13),
- `i` je pořadové číslo úlohy v rámci cvičení (1, 2, ...).

Příklady: `cv01_4` (4. úloha `cv01`), `cv12_58` (poslední úloha `cv12`).

3.3. Struktura `results`

Pole `results` drží **kolekci pojmenovaných výsledků**. Každý prvek pole reprezentuje jeden očekávaný vstup od studenta. Úloha tak může mít více polí (např. $D(f) = \dots$, *intervaly růstu*, *lokální extrém v $x = \dots$*).

Schema jednoho prvku:

```
{
  "key": "<identifikátor>",          # interní, pro logging
  "label_latex": "<LaTeX>",          # prefix před vstupem, např. "D(f) = "
  "type": "<typ>",                   # decimal | mathlive | multiple_choice |
    open_text
  "expected": <hodnota>,             # podle typu (viz níže)
  "tolerance": <float>,              # pouze pro type=decimal
  "options": [...],                 # pouze pro type=multiple_choice
}
```

3.4. Typy výsledků

Typ	expected	Vyhodnocení
decimal	float	MathLive vstup parsovaný přes Compute Engine <code>.N().valueOf()</code> ; porovnání numericky s <code>tolerance</code> .
mathlive	string (LaTeX)	Compute Engine 3-fázová strategie: <code>canonical.isSame()</code> → <code>simplify().isSame()</code> → <code>.N()</code> numericky.
multiple_choice	klíč správné možnosti	Radio buttony; <code>options[]</code> je pole <code>{key, label_latex}</code> .
open_text	vzorové řešení (string)	Žádné auto-vyhodnocení; doporučuje LLM (Gemini) nebo expertní revizi.

4. Vektor znalostí a kategorie

4.1. Dvě úrovně anotace úlohy

U každé úlohy rozlišujeme **dvě** ortogonální věci:

1. **Vektor znalostí** (`knowledge_vector`, 65 vah) — celý mix vlastností, typu, dovedností i kategorií SŠ/VŠ. Slovník přiřazuje každé skill-komponentě procentuální váhu (0–100). Při řešení úlohy se přírůstek (úspěch) nebo úbytek (nesplnění) studentovy znalosti rozdělí mezi komponenty v poměru těchto vah. Paralelní vektor existuje i ve studentově profilu — tam ale hodnoty $\in [0, 1]$ (IRT/BKT).
2. **Anotace úlohy** (uložená „bokem“ pro pozdější automatické předvyplnění `knowledge_vector`):
 - `category` — jedna primární kategorie ze 45 (jen SŠ + VŠ matematika + Výroková logika).
 - `properties` — multi-select 11 vlastností.
 - `task_type` — multi-select 2 typů (*Aplikační, Extrémy funkcí*).
 - `skills` — multi-select 6 dovedností.

Anotace nemá žádný runtime efekt v IRT/BKT — slouží jen k tomu, aby z ní engine v další fázi vygeneroval výchozí distribuci vah v `knowledge_vectoru`, kterou expert ručně doladí.

4.2. KNOWLEDGE_WEIGHTS (65 vah, verze 260820)

Definováno v `tasks/knowledge_weights.py`. Seznam pochází z konzultace s dr. Medkovou (základ 260601_v2, revize 2026-07-31 — viz *Změny 2026-07-31* na konci sekce). Obsahuje pouze *listové* váhy — hlavní (header) kategorie z původního excelu nejsou v seznamu. Všechny váhy jsou seskupené do 14 *skupin* pro účely barevného rozlišení v UI a budoucí paušální distribuce.

Skupiny vah (14). Většina vah nese prefix tvaru Skupina - Název (např. "Limita - LVL (krácení)"). Položky bez prefixu (*Konvexnost/konkávnost, Průběh funkce, Monotonie a extrémy, Výroková logika*) mají vlastní samostatnou skupinu.

Skupina	#	Barva v UI (CSS hex)
Vlastnosti	11	zelená #2e7d32
Typ	2	tmavě oranžová #ef6c00
Dovednosti	7	červenooranžová #d84315
SŠ	5	olivově žlutá #8d6e00
Výroková logika	1	blue-gray #607d8b
Funkce	7	modrá #1565c0
Monotonie a extrémy	1	fialová #6a1b9a
Konvexnost/konkávnost	1	břidlicová #455a64
Spojitosť	2	tyrkysová #00695c
Limita	7	růžová #c2185b
Derivace	7	tmavě modrá #283593
Průběh funkce	1	uhlově šedá #424242
Primitivní funkce	5	tmavě červená #b71c1c
Určitý integrál	8	hnědá #4e342e
Celkem	65	

Barvy odpovídají třídám `.grp-*` v `templates/task_checker.html` a `templates/tasks_list.html`. Stejně barvy se používají třikrát:

1. v **textu labelu** každého slideru ve „Vektoru znalostí“ v editoru,
2. v **textu** `<option>` v select boxu *Kategorie úlohy*,
3. v **rámu** `cat-chip` ve sloupci *Kategorie* v seznamu úloh.

4.3. TASK_CATEGORIES, TASK_PROPERTIES, TASK_TYPES, TASK_SKILLS

KNOWLEDGE_WEIGHTS (64) je v `tasks/knowledge_weights.py` rozdělen do čtyř disjunktních anotovaných podmnožin:

- **TASK_CATEGORIES** (45) — SŠ + Výroková logika + Funkce + Monotonie a extrémy + Konvexnost + Spojitost + Limita + Derivace + Průběh funkce + Primitivní funkce + Určitý integrál. Použito jako *single-select* v UI: jedna primární kategorie úlohy.
- **TASK_PROPERTIES** (11) — skupina *Vlastnosti*. *Multi-select* (checkboxy).
- **TASK_TYPES** (2) — skupina *Typ* (*Aplikační*, *Extrémy funkcí*). *Multi-select*.
- **TASK_SKILLS** (6) — skupina *Dovednosti*. *Multi-select*.

Součet $45 + 11 + 2 + 6 = 64$ pokrývá celý KNOWLEDGE_WEIGHTS (sanity-check při importu modulu).

V další fázi (zatím **není implementováno**) bude každá kategorie + kombinace tagů mít defaultní distribuci vah, která se automaticky aplikuje na `knowledge_vector` při anotaci úlohy, a expert ji pak doladí ručně.

4.4. Kompletní seznam vah

Plný seznam 65 vah, seskupený dle prefixu. Pořadí v tabulce odpovídá pořadí ve zdrojovém kódu i v UI.

Poznámka: tabulka níže je snapshot z revize 2026-07-31; *single source of truth* je `tasks/knowledge_weights.py` (obsahuje sanity-check assertions, které selžou, pokud spočtené počty nesouhlasí). Detailní změny oproti verzi 260601_v2 shrnuje závěrečná subsekcce.

#	Název	Stručná charakteristika
<i>Vlastnosti (9)</i>		
1	Vlastnosti - Lineární	Manipulace s lineárními výrazy a funkcemi $ax + b$.
2	Vlastnosti - Kvadratická	Kvadratické výrazy $ax^2 + bx + c$, kořeny, vrchol.
3	Vlastnosti - Mocninná	Mocninné funkce x^n , pravidla pro mocniny.
4	Vlastnosti - Logaritmická	Logaritmy a jejich vlastnosti.
5	Vlastnosti - Exponenciální	Exponenciální funkce a^x, e^x , vzorce.
6	Vlastnosti - Goniometrická	$\sin, \cos, \tan, \cot$ a jejich vzorce.
7	Vlastnosti - Absolutní hodnota	$ x $, rozklad podle znaménka.
8	Vlastnosti - Lomená lineární	Funkce $\frac{ax+b}{cx+d}$.
9	Vlastnosti - Racionální funkce	Racionální funkce (podíly polynomů). Přejmenováno z „Zlomky“ 2026-08-20.
<i>Typ (1)</i>		
10	Typ - Aplikační	Slovní / aplikační úlohy, modelace reálné situace.
<i>Dovednosti (7)</i>		
11	Dovednosti - Aplikace vzorce	Klasické vzorce $(a \pm b)^2, a^2 - b^2$, kosinová věta atd.
12	Dovednosti - Vytýkání, krácení	Algebraické zjednodušování, faktorizace.
13	Dovednosti - Roznásobení závorek	Distributivní zákon $a(b + c) = ab + ac$.
14	Dovednosti - Výpočet rovnic	Manipulace s rovnicemi (převody, substituce).
15	Dovednosti - Výpočet nerovnic	Řešení nerovnic, znaménková analýza.
16	Dovednosti - Derivování	Aplikace derivačních pravidel jako mechanická dovednost.
17	Dovednosti - Úpravy zlomků	Úpravy racionálních výrazů (přidáno 2026-08-20).
<i>SŠ (5)</i>		

#	Název	Stručná charakteristika
17	SŠ - Algebraické výrazy	Úpravy, zjednodušování polynomiálních / racionálních výrazů.
18	SŠ - Elementární funkce	Vlastnosti základních funkcí bez derivace.
19	SŠ - Rovnice	Řešení rovnic vč. exp., log., goniometrických.
20	SŠ - Nerovnice	Řešení nerovnic s jednou neznámou.
21	SŠ - Soustavy rovnic	Lineární soustavy 2–3 neznámých (dosazovací / sčítací).
<i>Funkce (7)</i>		
22	Funkce - Definiční obor	Stanovení D_f (odmocniny, jmenovatele, log, arcsin).
23	Funkce - Aritmetika	Operace $f + g$, $f \cdot g$, f/g a jejich def. obory.
24	Funkce - Skládání/rozkládání	$f \circ g$, určení vnitřní / vnější funkce, iterace.
25	Funkce - Sudá/lichá	Symetrie funkce vzhledem k ose y resp. k počátku.
26	Funkce - Inverzní funkce	Stanovení f^{-1} , def. obor a obor hodnot.
27	Funkce - Tečna ke grafu	Rovnice tečny / normály v daném bodě.
28	Funkce - Asymptota	Vertikální, horizontální a šikmé asymptoty.
<i>Monotonie (2)</i>		
29	Monotonie - Absolutní extrémů na intervalu	Globální max/min na uzavřeném intervalu.
30	Monotonie - Určování lokálních extrémů	Lokální max/min z první (a druhé) derivace.
<i>Konvexnost/konkávnost (1)</i>		
31	Konvexnost/konkávnost	Intervaly konvexity, konkávy z f'' , inflexní body.
<i>Spojitosť (2)</i>		
32	Spojitosť - Spojitosť	Spojitosť v bodě, na intervalu; dodefinování.
33	Spojitosť - Bolzanova věta	Existence kořene v uzavřeném intervalu.
<i>Limita (7)</i>		
34	Limita - VOAL (dosazení)	Věta o aritmetice limit — dosazení do spojitě funkce.
35	Limita - LVL (krácení)	Typ 0/0 řešený krácením společného činitele.
36	Limita - S odmocninou	Typ 0/0 řešený rozšířením odmocninou.
37	Limita - Vytknutí nejvyšší mocniny	Typ ∞/∞ v $\pm\infty$.
38	Limita - Jednostranné limity	$\lim_{x \rightarrow a^\pm}$, neexistence dvoustranné limity.
39	Limita - Typová limita	Tabulkové limity ($\sin x/x$, $(e^x - 1)/x$, ...).
40	Limita - Lhopitalovo pravidlo	L'Hospitalovo pravidlo pro 0/0 a ∞/∞ .
<i>Derivace (7)</i>		
41	Derivace - Sčítání	Pravidlo $(f + g)' = f' + g'$.
42	Derivace - Součin	Pravidlo $(fg)' = f'g + fg'$.
43	Derivace - Podíl	Pravidlo $(f/g)' = (f'g - fg')/g^2$.
44	Derivace - Složená funkce	Řetězkové pravidlo $(f \circ g)' = f'(g) \cdot g'$.
45	Derivace - Diferenciál	$df(x, h) = f'(x) \cdot h$; aproximace, odhady chyb.
46	Derivace - Taylorův polynom	Taylorův / Maclaurinův polynom řádu n .
47	Derivace - Vyšší řády	$f^{(n)}$ a jejich výpočet.
<i>Průběh funkce (1)</i>		
48	Průběh funkce	Komplexní vyšetření průběhu (D , parita, monotonie, ...).
<i>Primitivní funkce (5)</i>		
49	Primitivní funkce - Sčítání	Linearita integrálu.
50	Primitivní funkce - Per partes	$\int u dv = uv - \int v du$ pro neurčitý integrál.

#	Název	Stručná charakteristika
51	Primitivní funkce - 1.věta o substituci	$\int f(g(x))g'(x) dx = \int f(t) dt$.
52	Primitivní funkce - 2.věta o substituci	Substituce $x = \varphi(t)$ (typicky s odmocninou nebo exp.).
53	Primitivní funkce - Parciální zlomky	Rozklad racionální funkce, integrace.
<i>Určitý integrál (8)</i>		
54	Určitý integrál - Sčítání	Linearita určitého integrálu.
55	Určitý integrál - Aditivita	$\int_a^c f = \int_a^b f + \int_b^c f$ pro $a < b < c$.
56	Určitý integrál - Per partes	Per partes pro určitý integrál (s mezemi).
57	Určitý integrál - 1.věta o substituci	První věta s úpravou mezí.
58	Určitý integrál - 2.věta o substituci	Druhá věta s úpravou mezí.
59	Určitý integrál - Parciální zlomky	Parciální zlomky v určitém integrálu.
60	Určitý integrál - Nevlastní	Integrály s nekonečnými mezemi nebo neomezenou funkcí.
61	Určitý integrál - Obsah plochy	Výpočet obsahu rovinné oblasti ohraničené grafy funkcí.

Pozn. k drobným typografickým odchylkám. V seznamu se opakovaně objevuje „Vlastnosti“(1× „Vlastnosti“pro Goniometrickou) a „substituci“místo „substituci“— ponecháno přesně dle excelu dr. Medkové, ať se interní názvy nerozcházejí mezi zdroji.

4.5. Změny 2026-07-31 (v1.2)

Revize dle zpětné vazby dr. Medkové:

- **Sloučení:** *Monotonie - Absolutní extrém na intervalu + Monotonie - Určování lokálních extrémů* → *Monotonie a extrém* (top-level, mimo prefix). Skupina *monotonie* má nyní 1 položku (dříve 2).
- **Přejmenování:**
 - *Limita - Typová limita* → *Limita - Limita složené funkce*.
 - *Primitivní funkce - Parciální zlomky* → *Primitivní funkce - Racionální funkce*.
 - *Určitý integrál - Parciální zlomky* → *Určitý integrál - Racionální funkce*.
 - *SŠ - Výroková logika* → *Výroková logika* (vyňata ze skupiny SŠ, nová samostatná skupina *logika* s barvou #607d8b).
- **Přidané položky** (během roku 2026):
 - *Vlastnosti - S parametrem*
 - *Vlastnosti - Odmocninová*
 - *Typ - Extrémy funkcí*
- **Celkové počty:** KNOWLEDGE_WEIGHTS 61 → 64, TASK_CATEGORIES 45 (bez změny, ale s jiným složením), TASK_PROPERTIES 9 → 11, TASK_TYPES 1 → 2, skupin 13 → 14.
- **DB migrate:** 107 řádků `math_tasks.category` přepsáno v jedné transakci (Monotonie merge 39, Limita 22, Primitivní 11, Určitý integrál 0, Výroková logika 35). Zálohy `pg_dump + JSON` před migrací uloženy do `~/adaptmath_backups/` (s timestampy 2026-07-31 14:11, 14:55, 15:10).

Konvence task_id: zero-pad. Zároveň s taxonomií revidováno i číslování úloh — příklad se zapisuje *dvouciferně* (cv03_02, cv11_04, ...) místo dřívějšího cv03_2 / cv11_4. Sjednocení řeší seřazení v seznamu úloh (dříve cv01_1, cv01_10, cv01_11, ..., cv01_2, cv01_3). Migrace přepsala 117 řádků (116 přeformátovaných + 1 recovery úlohy s prázdným task_id). Backend v /api/tasks/<id> POST teď odmítá prázdný nebo whitespace-only task_id (HTTP 400), aby se stejný incident nemohl opakovat.

4.6. Změny 2026-08-20 (v1.3)

Velká iterace: extrakce dalších úloh z UMAT skriptu + Overleaf sbírky, revize taxonomie a hromadné čištění anotací.

Nové úlohy: +508 (DB 415 → 923).

- **Overleaf SŠ sbírka Andrei** (172 úloh, prefix ss01..ss07): Analytická geometrie (55), Goniometrie (41), Množiny (9), Rovnice s abs. hodnotou (5), Složené funkce (12), Úpravy alg. výrazů (43), Zjednodušte výraz (7). Extraktor v jedné dávce, žádný duplikát.
- **UMAT skriptum 2007-05-29** (336 úloh): první parser scripts/extract_umat.py zvládl přímé \uloha/\podul makra — 196 úloh (prefix umat_06..umat_09: Goniometrické rovnice, Posloupnosti, Limita funkce, Derivace). Druhý parser scripts/extract_umat_v2.py přidal rozklad \begin{ul} bloků + Řešení odstavce + konverzi textových odpovědí na MC — 140 úloh (prefix e01,e03,e04,e10,e11 = extra kapitoly Logika, Rovnice, Reálné funkce, Optimalizace, Primitivní funkce). Prefix eXX_NN použit záměrně, aby task_id nekolidovalo s umat_06..09 (chráníme starý index pro potenciální anotace).
- Skript scripts/insert_new_tasks.py pro idempotentní vkládání — jen INSERT nových task_id, nikdy DELETE/UPDATE existujících.

Taxonomie: Zlomky → Racionální funkce, +Úpravy zlomků.

- *Vlastnosti - Zlomky → Vlastnosti - Racionální funkce.* DB migrace přepsala 156 řádků v properties. Kód knowledge_weights.py aktualizován.
- **Přidáno:** *Dovednosti - Úpravy zlomků* (prázdná, čeká na tagování týmem).
- **Celkové počty:** KNOWLEDGE_WEIGHTS 64 → 65, TASK_SKILLS 6 → 7 (Vlastnosti bez změny počtu, jen rename), TASK_CATEGORIES 45 (bez změny).

České goniometrické funkce v MathLive UI.

- Custom window.mathVirtualKeyboard.layouts v task_checker.html — přepsán default MathLive keyboard, tak aby student viděl české tg, cotg, arctg, arccotg místo anglických tan, cot, arctan, arccot. Layout f(x) obsahuje i další užitečná tlačítka (lim, ∫, d/dx, Σ, ∞). Layout ∞ ∉ je také custom (bez defaultních inverse-trig tlačítek). Pořadí tabů: 123 / f(x) / αβγ / ∞ ∉, čísla jsou výchozí aktivní.
- Registrované macros MathfieldElement.macros: \tg → \tan, \cotg → \cot, atd. Compute Engine tak sémanticky porovnává české a anglické zápisy jako totožné (obousměrná ekvivalence). Existující úlohy s \tan v expected fungují bez migrace.
- Bulk migrace content_latex + results[*].expected: \tan/\cot/\arctan → \tg/\cotg/\arctg (word-boundary regex chrání před \tanh). Přepsáno 45 řádků. Skript scripts/migrate_trig_to_czech.py.

CSS: overflow fix v preview boxu. Dlouhé LaTeX expression bez soft-breaks už nepřetéká — .preview-box má overflow-x: auto + max-width: 100%, .katex-display má vlastní horizontální scroll.

Bulk čištění text-answer úloh (105 opravených). Auditem se identifikovalo 112 e-/umat-úloh, u kterých parser nechal `expected` jako string začínající písmenem (parser to považoval za text). Skript `scripts/fix_text_answers_batch.py` zpracoval 4 kategorie:

- **PURE_MATH** (80 úloh): `expected` byl `math` bez obalu `$.$.` (např. `x=-2`). Wrap do `$.$.` + expand UMAT-specific maker (`\Ra` → `\Rightarrow`). Typ zůstává `mathlive`.
- **PURE_TEXT** (11 úloh): čisté texty (negace výroků z e01, „prostá, důkaz sporem“, „neex.“). Konverze na `multiple_choice` s ručně navrženými pedagogicky rozumnými distraktory.
- **MIXED_TEXT_MATH** (4 úlohy): „není prostá: např. `$f(x)=f(y)$` pattern → MC.
- **COMPOUND** (10 úloh): více `math` hodnot v jednom `expected` — split na keys (`a/v`, `k/s_8`, `a1/q`) nebo MC pro edge cases.

Skipnuto: 1 úloha (e03_35 — megablok 9 nerovnic v jednom, čeká na ruční rozdělení).

Cleanup e-úloh: layout junk. 20 e-úloh mělo v `expected` zbylé LaTeX layout artefakty (`\clubsuit`, `\vspace`, `\pagebreak`, `\section`, osiřelé `\` nebo `\,`). Skript `scripts/cleanup_e_layout_junk.py` iterativně stripoval, bez ztráty smysluplných teček (věty typu „ani Eliška.“ zůstávají).

Post-audit fix (15 úloh dnes). Systematický scan `mathlive` výsledků odhalil 16 zbývajících úloh s komplikovanými zápisy (`Df/Hf` notace, multi-value, „`X = a` nebo `X = b`“ apod.). Opraveno:

- 5 úloh s `$Df = ...$` → label `Df =`, `expected` jen interval (`mathlive`).
- 1 úloha s `$x=12\text{ cm}$` → `decimal 12` + label.
- 2 úlohy se 2 čísly → split na 2 decimal keys.
- 7 úloh s komplexním pattern (`Df` s podmínkami, slovní úlohy, 2 řešení GP/AP) → konverze na `multiple_choice` se 4 pedagogickými distraktory.

Split komplexních results. Několik iterací per-task úprav:

- Parabola úlohy (e04_39..e04_44, 6 úloh): 1 `mathlive` „osa: `x = ..`, `V[...]`“ → 3 samostatné keys `osa`, `Vx`, `Vy`.
- Kořeny (e03_04..e03_11, 6 úloh): 1 `mathlive` „`x1 = ..`, `x2 = ..`“ → 2–3 keys `xi`.
- F-funkce (e04_25..e04_30, 6 úloh): 1 `mathlive` „`F1 = ..`, ..., `F4 = ..`, `DF1.. = ..`“ → 4 `mathlive` keys (`Fi`) + 1 `multiple_choice` key (`D`) se 4 možnostmi pro definiční obory (protože komplexní compound `Df` zápisy jsou pro `mathlive` nevhodné).
- Inverze funkce (e04_31..e04_38, 8 úloh): 1 `mathlive` „`f : y = ..`; `Df-1 = ..`; `Hf-1 = ..`“ → 3 samostatné `mathlive` keys `y`, `Df_inv`, `Hf_inv`.

Nové skripty. `scripts/` obsahuje: `extract_umat.py`, `extract_umat_v2.py` (UMAT extrakce), `insert_new_tasks.py` (idempotentní INSERT), `migrate_trig_to_czech.py`, `cleanup_e_layout_junk.py`, `fix_text_answers_batch.py`, `llm_generate_mc.py` (Claude API integrace pro budoucí generování MC). Adhoc SQL migrace + Python `/tmp/*.py` skripty pro per-task fixy (parabola, kořeny, F-funkce, inverse, D-key MC, e04_44 redesign, atd.) jsou zdokumentovány v git history commitů.

DB zálohy: 12 snapshotů dnes. Před každou skupinou zásahů proveden `pg_dump` + JSON snapshot, stažené i lokálně (`.backups/` s SHA256 ověřením). Timestamps: 20260731_141115, 145549, 151025; 20260803_131727, 144849, 151050; 20260820_081742, 083207, 090717, 100539, 165023, 171817. Restore-recept: `pg_restore -c -d $DATABASE_URL <path>.dump`.

4.7. Změny 2026-08-24 (v1.4): Pilotní IRT seed

Otevřeno lektorům ruční nastavování IRT obtížnosti přes UI, aby se dala rozjet adaptivní selekce úloh v pilotu.

UI: select IRT obtížnost v editoru úlohy. V `templates/task_checker.html` přibyl v sekci „Identifikace & klasifikace“ nový `<select>` *IRT obtížnost* se třemi předdefinovanými hodnotami:

- **Lehká** $\rightarrow b = -2$
- **Střední** $\rightarrow b = 0$ (výchozí)
- **Těžká** $\rightarrow b = +2$

Two-way binding do `state.irt_difficulty` (jako `float`), autosave přes stejný POST endpoint jako ostatní pole úlohy. Když je v DB NULL nebo jiná hodnota, UI mapuje na nejbližší z $\{-2, 0, +2\}$.

Backend validace v app.py. Endpoint POST `/api/tasks/<id>` nyní pro klíč `irt_difficulty` přijímá jen tři povolené hodnoty $(-2.0, 0.0, +2.0)$; jiná hodnota vrátí HTTP 400. Diskriminaci (`irt_discrimination`) přes UI needitujeme.

DB backfill. Před migrací fresh backup `adaptmath_20260824_135958.dump` + JSON snapshot (staženo lokálně, SHA256 ověřeno). Transakční migrace:

- UPDATE `math_tasks` SET `irt_difficulty` = 0.0 WHERE `irt_difficulty` IS NULL $\rightarrow$ 923 úloh.
- UPDATE `math_tasks` SET `irt_discrimination` = 1.0 WHERE `irt_discrimination` IS NULL $\rightarrow$ 923 úloh.

Po migraci ověřeno: `diff_null` = 0, `disc_null` = 0, všechny hodnoty v očekávaných mantinelech.

Proč $a = 1.0$, ne $a = 0$? V 2PL modelu $P(\text{správně}) = \sigma(a(\theta - b))$ je $a = 0$ degenerace: pravděpodobnost správné odpovědi je pak vždy 0,5 bez ohledu na θ i b . Úloha „nediskriminuje“ a b ztrácí smysl. Hodnota $a = 1.0$ je standardní Rasch/2PL default („úloha rozlišuje neutrálně“) a je vhodný startovní bod pro empirickou rekaliibraci přes `py-irt` po nasbírání dat od studentů (viz IRT.md, kapitola 4).

Dopad na adaptivní selekci. Selektor podle $|b - (\theta + \Delta_{\text{Vygotsky}})|$ nyní dostává reálné hodnoty místo NULL a může začít cílit obtížnost úloh podle BKT profilu studenta. Do doby empirické rekaliibrace zůstává selekce „hrubo-zrná“ (tři vrstvy obtížnosti), což je pro pilot dostatečné.

5. Webové UI: Task Checker

5.1. Routes

Metoda	Cesta	Účel
GET / POST	<code>/login</code>	Formulář pro zadání hesla (session-based ochrana).
GET / POST	<code>/logout</code>	Smaže session, redirect na <code>/login</code> .
GET	<code>/</code>	Redirect na první úlohu (vyžaduje login).
GET	<code>/tasks</code>	Tabulka všech úloh (filtr, checkbox výběr, bulk panel).
GET	<code>/tasks/<id></code>	Detail úlohy — editor + sandbox.
POST	<code>/api/tasks/<id></code>	Uložit změny (JSON body s editable poli).
POST	<code>/api/tasks/bulk</code>	Hromadná anotace vybraných úloh (viz níže).
DELETE	<code>/api/tasks/<id></code>	Smazat úlohu.

Odolnost proti `strict_slashes`. `app.url_map.strict_slashes = False` — Flask matchuje jak `/tasks`, tak `/tasks/`. Fix incidentu (Windows/Chrome uživatelům někdy doplňuje koncové lomítko).

5.2. Přihlášení a session

Všechny views kromě `/login` jsou chráněné dekorátorem `@require_login`. Při neautentizovaném přístupu se uživatel přesměruje na `/login` s parametrem `next` obsahujícím plnou URL původního požadavku (včetně reverse-proxy prefixu `/adaptmath/`). Po zadání správného hesla se vytvoří podepsaná session cookie a uživatel se vrátí na původní stránku.

Konfigurace přes env vars (systemd unit):

- `APP_PASSWORD` — heslo pro celou aplikaci (jeden globální účet pro tým),
- `FLASK_SECRET_KEY` — 64-znakový hex pro podepisování cookies,
- `APP_SESSION_DAYS` — platnost session v dnech (default 30).

Login screen má orange-accent design (logo „A“, brand *AdaptMath*, autofocus na input). V editoru je v hlavičce stránky odkaz *Odhlásit*.

5.3. Seznam úloh (`/tasks`)

Tabulka 415 úloh se sloupci: *Task ID*, *Náhled zadání* (KaTeX), *Typy výsledků* (badges), *Kategorie*, *Kognitivní zátěž*.

Sloupec Kategorie. Hodnota `t.category` z DB se zobrazí jako *cat-chip* (zaoblený rámeček) v barvě CSS třídy `.grp-XYZ` odpovídající skupině váhy. Pokud úloha kategorii ještě nemá nastavenou (typický stav po seedu), zobrazuje se světle šedý šrafovaný chip „— nepřiřazeno —“.

Filtr. Dropdown nad tabulkou nabízí všech 45 kategorií (text option barevně podle skupiny) plus speciální položku „— bez kategorie —“ pro rychlou identifikaci úloh, které tým ještě nezpracoval. Fulltext input hledá v `task_id`, hodnotě kategorie i v `content_latex`.

Hromadné tagování (bulk). Vlevo v každém řádku je checkbox, v hlavičce master checkbox „vybrat všechny aktuálně viditelné“ (respektuje filtr). Jakmile je vybrán ≥ 1 řádek, ve spodní části viewportu se objeví *sticky panel* se 4 sekcemi:

- *Kategorie* — mode select (**Nastavit vždy** / **Jen když chybí**) + select kategorií. Režim *Jen když chybí* přepíše pouze úlohy s NULL, existující hodnoty nemění.
- *Vlastnosti* / *Typ* / *Dovednosti* — mode select (**Přidat** / **Nahradit** / **Odebrat**) + rozklápěcí `<details>` s checkboxy. *Přidat* udělá union s existujícím výběrem, *Nahradit* nahradí celý list, *Odebrat* dropne uvedené hodnoty z existujícího výběru.

Před spuštěním se zobrazí potvrzovací *modal* se sumarizací operací a preview prvních 15 `task_ids`. Backend endpoint `POST /api/tasks/bulk` provede vše v jedné transakci a vrátí `{changed, skipped, not_found}`. Klávesa Esc nebo klik mimo modal ho zavře.

5.4. Editor úlohy (`/tasks/<id>`)

UI je rozděleno do dvou sloupců.

Levý sloupec (vizualizace + test).

- Živý KaTeX náhled zadání (re-renderuje při každém keystroke).
- Sandbox „Vyzkoušej řešení“ — MathLive vstupní pole; tlačítko *Otestovat* spustí vyhodnocení proti aktuálnímu (i neuloženému) **expected**.
- Sekce „Vzorové odpovědi“ — vyrenderované očekávané hodnoty.

Pravý sloupec (editor).

- Task ID, kognitivní zátěž (select A–F).
- *Kategorie úlohy* — single-select 45 položek (SŠ + Výroková logika + VŠ matematika), text barevně rozlišený podle příslušné skupiny.
- Pod selektem 3 *multi-select* sekce s checkboxy — jsou barevně sladěné se skupinou:
 - *Vlastnosti* (11 položek, zelená),
 - *Typ* (2 položky — Aplikační, Extrémy funkcí, krémová),
 - *Dovednosti* (7 položek, oranžová).

Hodnoty se ukládají do JSON sloupců **properties**, **task_type**, **skills**. Serializace důsledně rozlišuje NULL (nikdy neannotováno) a [] (vědomě prázdná anotace) — viz *carry button* níže.

- Zadání v LaTeXu (textarea, full-width).
- Editor výsledků — karta pro každý prvek pole **results**; lze přidat/odebrat; pro každý se mění **key**, **label_latex**, typ (select), a **expected** (specifické pro typ):
 - **decimal** — číselný input + tolerance.
 - **mathlive** — 1–N MathLive polí (viz *Multi-variant* níže).
 - **multiple_choice** — řádky s viditelným *oranžovým radiobuttonem* vlevo pro označení správné odpovědi (opraveno v revizi 2026-07-31, dřív bylo skryté kvůli generickému `.row input {width:100%}` pravidlu), key inputem, label inputem a mazacím tlačítkem.
 - **open_text** — textarea se vzorovým řešením (vyhodnocuje ručně).

Multi-variant MathLive expected. U typu **mathlive** umí **r.expected** být buď *string* (1 přijatelná varianta — backwards-compatible s existujícími úlohami), nebo *list stringů* (2 a více variant — student uspěje, pokud trefí kteroukoli přes Compute Engine). V UI je pod polem tlačítko „+ Přidat další správnou variantu“; přidáním druhé varianty se serializace přepne na list. Evaluátor zkouší varianty postupně (symbolic → simplified → numeric) a v ‘mode’ pole vrací index varianty, která uspěla. Řeší úlohy jako **cv12_28**, kde MathLive nerozpozná ekvivalenci mezi ekvivalentními zápisy s racionálními mocninami.

Autosave a carry button. Editor má *debouncovaný autosave* (800 ms po každé změně, indikátor „ukládám... / uloženo v HH:MM:SS“). Navigační tlačítka *Prev* / *Další* flushnou pending save před přechodem, **beforeunload** varuje při zavření tabu s neuloženými změnami. Vedle plain „Další →“ je oranžový button „*Další s kopií anotace* →“ (one-shot přes **sessionStorage**) — přenesení **category** + zaškrtnuté *Vlastnosti/Typ/Dovednosti* do následující úlohy, ale **jen do polí, která jsou v DB NULL**. Nikdy nepřepisuje existující ani vědomě prázdné []. Šetří klikání pro sekvenční úlohy stejné kategorie ve stejném cvičení.

Pod editorem (full-width) — Vektor znalostí.

- Plochý seznam **61 vah** ve 2 sloupcích; každý řádek má label v barvě skupiny, range slider (0–100, krok 1), spojený number input a znak %.
- Nad gridem je **legenda** 13 skupin — každá s barevnou tečkou a jménem skupiny.

- Filtr textový („lim“, „derivace“) + checkbox *jen nenulové*.
- Sumarizace v hlavičce sekce: aktuální součet všech vah, počet nenulových z 61, tlačítko *Vynulovat vše*.
- Slider $\leftrightarrow$ number input jsou dvousměrně synchronizované; aktivní řádky (váha > 0) jsou zvýrazněné a uložené jako jediné v JSON (chybějící klíč = 0).

Sticky save bar (dole).

- *Uložit změny* — POST /api/tasks/<id> s celým stavem;
- *Vrátit* — obnoví poslední uložený stav;
- *Smazat úlohu* — DELETE endpoint.

Klávesové zkratky. $\leftarrow$ a $\rightarrow$ pro navigaci mezi úlohami, Ctrl+S (resp. Cmd+S) pro uložení.

5.5. Tipy pro vyhodnocení MathLive Compute Engine

- `mathlive` odpovědi se ukládají jako čistý LaTeX. Compute Engine je tolerantní k variantám zápisu zlomku, znamének, `\infty` vs `+\infty`, atd.
- Pro intervaly preferujeme standardní zápis: `(a, b)` otevřený, `[a, b]` uzavřený, `\cup` pro sjednocení.
- Tam, kde by Compute Engine selhal (cyklické sjednocení po $k \in \mathbb{Z}$, „limita neexistuje“ jako popis), používáme `multiple_choice` se třemi variantami (1 správná + 2 typické chyby).

6. Datový obsah

6.1. Zdroj úloh

Skriptu *Základy matematiky 1* (P. Pražák, P. Bauer; KIKM FIM UHK). Zdrojové `.tex` soubory cvičení 1–13 byly ručně (LLM-asistovaně) interpretovány a převedeny do strukturovaného formátu v `tasks/cvNN.py`.

6.2. Pokrytí

Cvičení	Téma	Úloh
cv01	Funkce, definiční obor, kvadratická funkce	37
cv02	Skládání, prostá, inverzní funkce	29
cv03	Monotonie, parita, polynomy, Hornerovo schéma	42
cv04	Limita — aritmetika limit	18
cv05	Limita — věta o složené funkci, typové limity	44
cv06	Spojitosť funkce	23
cv07	Derivace (součet, součin, podíl, řetězkové pravidlo)	47
cv08	Tečna, normála, absolutní extrémy, L'Hospital	36
cv09	1. derivace, lokální extrémy, optimalizace	26
cv10	2. derivace (konvexita, inflexe), asymptoty	18
cv11	Diferenciál, Taylorův a Maclaurinův polynom	27
cv12	Primitivní funkce (linearita, per-partes, substituce)	58
cv13	Integrace racionálních funkcí	10
Celkem		415

6.3. Distribuce typů výsledků

Typ	Počet
mathlive	319
multiple_choice	115
decimal	83
open_text	0 *

*Původní negace výroků byly převedeny na multiple_choice se 3 možnostmi (správná + 2 typické chyby studentů).

7. Spuštění lokálně

7.1. Závislosti

- Python 3.10+
- Docker (pro PostgreSQL) — nebo systémový PostgreSQL 14+
- Python balíčky: flask, sqlalchemy, psycpg2-binary

7.2. První spuštění

```
# 1) klon repozitare
git clone https://github.com/dominikpalla/adapt_math.git
cd adapt_math

# 2) Python venv + dependencies
python3 -m venv .venv
source .venv/bin/activate
pip install flask sqlalchemy psycpg2-binary

# 3) Spustit PostgreSQL kontejner (lokálně)
docker compose up -d

# 4) Naplnit DB (drop & recreate + insert 415 uloh)
python seed_db.py

# 5) Spustit Flask
python app.py
```

Aplikace běží na <http://127.0.0.1:5000>. Přihlašovací heslo je default kikkjenejlepsi (env APP_PASSWORD).

7.3. Konfigurace přes env vars

Pro produkci preferujeme všechny citlivé hodnoty mimo zdrojový kód:

Env var	Účel
DATABASE_URL	Připojení k PostgreSQL (postgres://...).
APP_PASSWORD	Heslo pro Flask login screen.
FLASK_SECRET_KEY	Hex string pro podepisování session cookies.
APP_SESSION_DAYS	Doba platnosti session (default 30).

7.4. Opětovné nasazení po změně schématu

Jelikož v aktuální fázi nepoužíváme migrace (Alembic), po každé změně sloupců v `model.py` stačí spustit znovu `python seed_db.py` — skript provede `DROP TABLE ... CASCADE` a `CREATE TABLE` dle aktuálního ORM modelu, načte úlohy vloží znovu.

Pozor: drop & recreate ztrácí veškerá data nasazená přes UI (ručně upravené kategorie, vektory znalostí). Pro produkci je nutné zavést migrace.

8. Nasazení do produkce (UHK)

Aplikace je nasazena na školní VM na adrese:

<https://moodlefim.uhk.cz/adaptmath/>

Server je sdílený s existující Moodle instancí (FIM UHK, vnitřní IP 172.25.14.164, Apache 2.4 + MariaDB). AdaptMath běží *vedle* Moodleu jako Apache reverse proxy do lokální Flask aplikace, bez jakéhokoli zásahu do běhu Moodleu.

8.1. Architektura nasazení

1. **Flask aplikace** běží jako systemd služba `adaptmath.service` pod uživatelem `moodlepalla` a naslouchá pouze na 127.0.0.1:5000 (loopback, ven se nedostane).
2. **PostgreSQL 14** (systémový, ne Docker) běží na 127.0.0.1:5432 s databází `adaptmath` a uživatelem `adaptmath_user`.
3. **Apache vhost** `moodle-ssl.conf` dělá reverse proxy:

```
RedirectMatch permanent ^/adaptmath$ /adaptmath/
ProxyPreserveHost On
ProxyPass          /adaptmath/ http://127.0.0.1:5000/
ProxyPassReverse   /adaptmath/ http://127.0.0.1:5000/
RequestHeader set  X-Forwarded-Prefix "/adaptmath"
```

4. Flask aplikace používá `werkzeug.middleware.proxy_fix.ProxyFix`, který čte `X-Forwarded-Prefix` z hlavičky a automaticky prependuje prefix k výstupům `url_for()`, takže všechny linky v UI vedou na `/adaptmath/...`

8.2. Systemd unit

Soubor `/etc/systemd/system/adaptmath.service` (chmod 600 kvůli `FLASK_SECRET_KEY`):

```
[Unit]
Description=AdaptMath Task Checker (Flask)
After=network.target postgresql.service
Requires=postgresql.service

[Service]
Type=simple
User=moodlepalla
WorkingDirectory=/home/moodlepalla/adapt_math
Environment="DATABASE_URL=postgresql://adaptmath_user:***@localhost:5432/adaptmath"
Environment="APP_PASSWORD=***"
Environment="FLASK_SECRET_KEY=<64 hex chars>"
Environment="APP_SESSION_DAYS=30"
ExecStart=/home/moodlepalla/adapt_math/.venv/bin/python -m flask --app app
run --host 127.0.0.1 --port 5000
Restart=on-failure
```

```
RestartSec=3

[Install]
WantedBy=multi-user.target
```

8.3. Provozní příkazy

Akce	Příkaz
Status	<code>sudo systemctl status adaptmath</code>
Logy (live)	<code>sudo journalctl -u adaptmath -f</code>
Restart	<code>sudo systemctl restart adaptmath</code>
Update kódu	<code>cd ~/adapt_math && git pull && sudo systemctl restart adaptmath</code>
Změnit heslo	edit <code>adaptmath.service</code> , pak <code>daemon-reload + restart</code>
Reset DB	<code>DATABASE_URL=... .venv/bin/python seed_db.py</code>
Vrátit vhost	<code>sudo cp moodle-ssl.conf.bak.* moodle-ssl.conf + reload apache2</code>
Backup DB	<code>pg_dump "\$DATABASE_URL"-Fc -f ~/adaptmath_backups/adaptmath_\$(date +%Y%m%d_%H%M%S).dump</code>
Restore DB	<code>pg_restore -c -d "\$DATABASE_URL" ~/adaptmath_backups/<...>.dump</code>
Report XLSX	<code>DATABASE_URL=... .venv/bin/python scripts/export_task_stats.py -out ...</code>

Export statistik tagů. Skript `scripts/export_task_stats.py` vytvoří komplexní XLSX (9 listů: Souhrn, Kategorie, Vlastnosti, Typy, Dovednosti, Kategorie×Vlastnosti, Kategorie×Dovednosti, Seznam úloh, Neanotované) s barevným rozlišením dle skupin. Jen *read-only* dotazy na DB. Slouží pro reporting směrem k dr. Medkové. Vyžaduje `openpyxl` v prod venvu.

8.4. Známá omezení

- **Werkzeug dev server** — aktuálně Flask běží přes `flask run`, což je vývojový server. Pro skutečnou produkci by bylo lepší přepnout na `gunicorn` (jednoduchá změna v `ExecStart`).
- **SSL certifikát** — ručně instalovaný v `/etc/ssl/certs/moodlefim.crt`, vyžaduje pravidelnou obnovu (řeší IT UHK; není Let's Encrypt s auto-renew).
- **Žádné DB migrace** — viz „Opětovné nasazení po změně schématu“ výše.
- **Jediný globální účet** — všichni členové týmu sdílí jedno heslo. Pro plnohodnotný multi-user režim budeme potřebovat Flask-Login a tabulku `users`.

9. Co dál (roadmap)

9.1. Krátkodobé

1. **Tagování úloh experty.** Tým musí manuálně projít 415 úloh a přiřadit `category` + nastavit `knowledge_vector`. To je hlavní úkol Task Checkeru.
2. **Distribuce vah dle kategorie.** Definovat slovník `CATEGORY_DEFAULT_VECTOR`, který pro každou ze 45 kategorií předvyplní výchozí distribuci 65 vah. Při výběru kategorie v UI se předvyplní `knowledge_vector`, expert pak ladí.
3. **LLM vyhodnocení open_text.** Aktuálně máme všechny negace převedeny na `multiple_choice`; pro budoucí volné úlohy zavést endpoint `POST /api/eval-open-text` volající Gemini.
4. **Alembic migrace.** Nahradit `drop & recreate` řízenými migracemi.
5. **Gunicorn pro produkci.** Nahradit Werkzeug dev server na produkci.

9.2. Střednědobé

1. **IRT engine** (py-irt). Pilotní tréninková fáze pro kalibraci obtížnosti a diskriminace. Vlastní route `/student/<id>/next-task`, která vybere úlohu podle aktuálního θ studenta.
2. **BKT engine** (py-bkt). Po každé interakci aktualizuje studentův vektor znalostí na základě `knowledge_vector` úlohy a parametrů α , β .
3. **Studentský profil**. Tabulka `students` s preferencemi (styl učení, motivace, math anxiety) a vektorem znalostí (64 hodnot v $[0, 1]$).
4. **Multi-user režim**. Tabulka `users` + Flask-Login pro plnohodnotné přihlašování více účtů (zatím sdílíme jedno heslo).
5. **Logování interakcí** pro výzkum (čas řešení, jistota, AI nápověda).

9.3. Dlouhodobé

1. **Integrace s Moodle**. Engine jako externí Moodle plugin / LTI tool. Výstup TQ23000206-V3.
2. **Generování úloh LLM**. Tvorba variant na téma „kategorie + obtížnost“.
3. **Dashboard pro experty**. Vizualizace pokroku jednotlivých studentů, analýza odchylek od průměru.

10. Reference

- Repozitář projektu: https://github.com/dominikpalla/adapt_math
- Skripta *Základy matematiky 1*, P. Pražák, P. Bauer, FIM UHK
- MathLive: <https://github.com/arnog/mathlive>
- Compute Engine: <https://cortexjs.io/compute-engine/>
- KaTeX: <https://katex.org/>
- py-irt: <https://github.com/nd-ball/py-irt>
- py-bkt: <https://github.com/CAHLR/pyBKT>

Tato dokumentace byla vygenerována ve fázi plnění databáze úloh (Task Checker, verze 1.0). Bude rozšířena po implementaci IRT/BKT engine a integrace s Moodle.